

MÓDULO 8 — FINAL

Proyecto Final

• Sesión de Cierre

Sesión Final · 2 horas — try/except · Archivos · Módulos · Proyecto



Última sesión del curso — ¡Lo lograste!

Manejo de errores – try / except

Los errores son inevitables. try/except permite capturarlos y responder de forma controlada.

✗ Sin manejo de errores

```
edad = int(input("Edad: "))  
# El usuario escribe 'abc'  
# → ValueError: invalid literal  
# EL PROGRAMA SE CAE ✗
```

✓ Con try/except

```
try:  
    edad = int(input("Edad: "))  
except ValueError:  
    print("Ingresa un número.")  
# El programa SIGUE funcionando ✓
```

Estructura completa:

try:

1. Código que podría fallar
resultado = 10/divisor

except:

2. Si falla con ese error
ZeroDivisionError

else:

3. Si NO hubo error
print(resultado)

finally:

4. SIEMPRE se ejecuta
"Operación terminada"

Errores más comunes en Python

Conocer los tipos de error te permite capturarlos de forma específica y dar mensajes útiles.

ZeroDivisionError

```
10 / 0
```

División entre cero

ValueError

```
int('abc')
```

Valor inapropiado para el tipo

TypeError

```
'5' + 3
```

Tipos incompatibles

KeyError

```
dict['x']
```

Clave inexistente

IndexError

```
lista[99]
```

Índice fuera de rango

FileNotFoundError

```
open('x.txt')
```

Archivo inexistente

AttributeError

```
int.append(5)
```

Método inexistente

NameError

```
print(x_undef)
```

Variable no definida



Captura siempre el error más específico primero. `except Exception as e` al final como red de seguridad.

Leer y escribir archivos de texto

`open()` abre archivos. Siempre usa `'with'` para garantizar el cierre automático.

`'r'` Solo lectura

`'w'` Sobreescribe

`'a'` Agrega al final

`'x'` Crea exclusivo

```
# ESCRIBIR
with open('notas.txt','w',encoding='utf-8') as f:
    f.write("Ana López,88.5\n")
    f.write("Carlos Ramos,72.0\n")

# AGREGAR sin borrar
with open('notas.txt','a',encoding='utf-8') as f:
    f.write("María Quispe,95.0\n")

# LEER línea por línea
with open('notas.txt','r',encoding='utf-8') as f:
    for linea in f:
        linea = linea.strip()
        nombre, nota = linea.split(',')
        print(f"{nombre}: {nota}")
```

```
# MANEJO SEGURO de archivos
try:
    with open('notas.txt','r',encoding='utf-8') as f:
        contenido = f.read()
except FileNotFoundError:
    print('El archivo no existe')
except PermissionError:
    print('Sin permisos para leer')
```

⚠ Usa siempre `encoding='utf-8'` para evitar problemas con ñ, á, é en Windows.

Módulos estándar — os · datetime · json

Python incluye módulos listos para usar. Los más útiles para el proyecto final:

import os

```
os.path.exists('f.txt')
os.getcwd()
os.makedirs('datos',
    exist_ok=True)
os.path.join('datos', 'f.json')
```

import datetime

```
from datetime import datetime, date
ahora = datetime.now()
ahora.strftime('%d/%m/%Y %H:%M')
hoy = date.today()
hoy.year, hoy.month, hoy.day
```

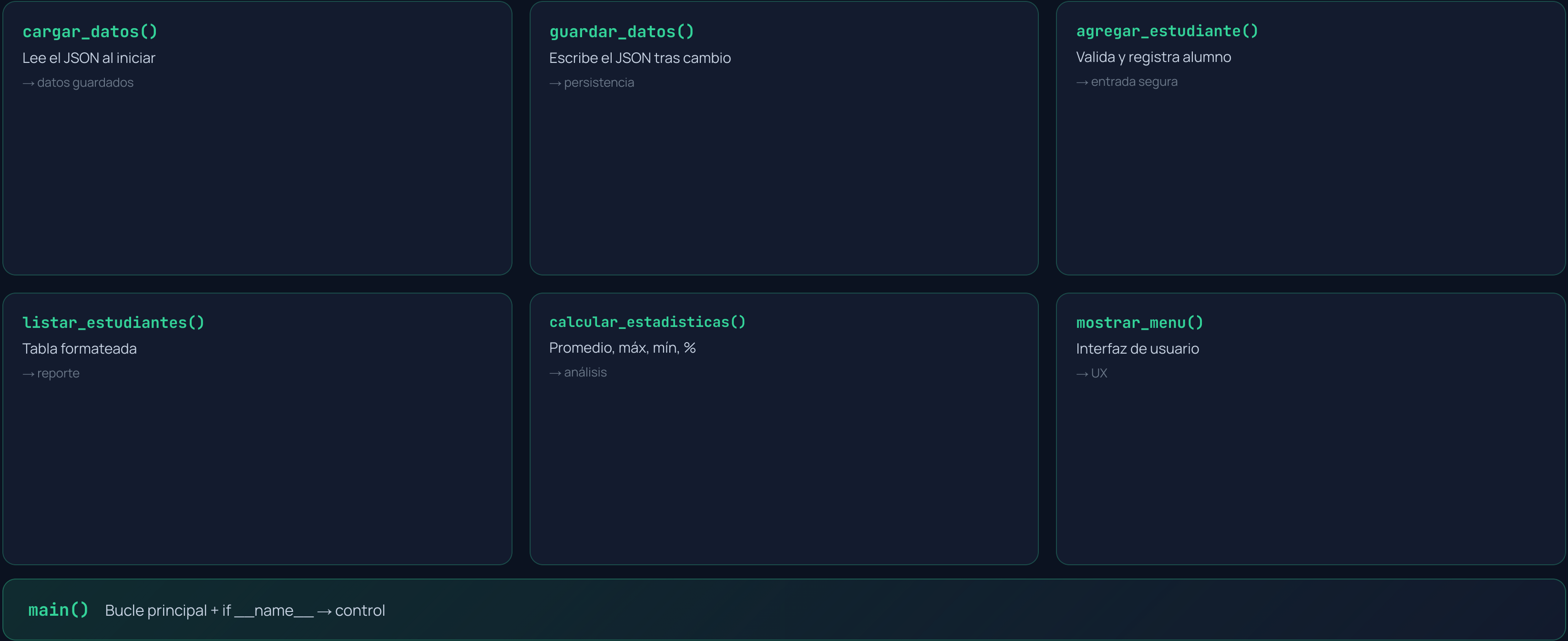
import json

```
# Dict → archivo
json.dump(datos, f, indent=4)
# Archivo → Dict
datos = json.load(f)
```

💡 if __name__ == '__main__': permite que el código solo se ejecute directamente, no al importar como módulo.

Proyecto final – Arquitectura del sistema

 Sistema de Registro de Estudiantes – Estructura modular y organizada



Proyecto final — Código principal

Fragmentos clave del sistema — el código completo está en el material del módulo.

```
def agregar_estudiante(estudiantes):
    try:
        nombre = input('Nombre: ').strip()
        if not nombre: raise ValueError("Nombre vacío")
        mat = input('Matrícula: ').upper()
        if mat in estudiantes: raise ValueError("Ya existe")
        nota = float(input('Nota: '))
        if not 0 ≤ nota ≤ 100: raise ValueError("Nota inválida")
        estudiantes[mat]={'nombre':nombre,'nota':nota,
            'estado':'Aprobado' if nota ≥ 51 else 'Reprobado',
            'fecha':datetime.now()...}
        guardar_datos(estudiantes)
    except ValueError as e: print(f"Error: {e}")

# JSON — cargar y guardar
def cargar_datos():
    if os.path.exists(ARCHIVO): return json.load(open(ARCHIVO,'r',encoding='utf-8'))
    return {}
def guardar_datos(est): json.dump(est,open(ARCHIVO,'w',encoding='utf-8'),ensure_ascii=False,indent=4)
```

```
def main():
    estudiantes = cargar_datos()
    while True:
        mostrar_menu()
        op = input('Opción: ')
        if op=="1": agregar_estudiante(est)
        elif op=="2": listar_estudiantes(est)
        elif op=="3": calcular_estadisticas(est)
        elif op=="0":
            print("¡Hasta luego!"); break
        else: print("Inválida")

if __name__=="__main__":
    main()
```

Repaso completo del curso — 8 módulos

Todo lo que aprendiste en Python Básico — ¡de cero a un sistema funcional!

M1 · Introducción

Python, VS Code, print(), terminal

M2 · Variables

int, float, str, bool, input(), type()

M3 · Operadores

+ - * / % **, ==, and/or, f-strings

M4 · Control

if/elif/else, while, for, break, continue

M5 · Funciones

def, params, return, scope, docstrings

M6 · Listas

[], índices, slicing, append, sort, tuplas

M7 · Dicts/Sets

{clave:valor}, .items(), set, |&-

M8 · Proyecto

try/except, archivos, json, os, datetime

Próximos pasos en Python

Python Básico es solo el comienzo. Estos son los caminos que puedes tomar:



Python Intermedio

OOP, decoradores, generadores, lambda, list comprehension



Ciencia de Datos

NumPy, Pandas, Matplotlib, scikit-learn, Jupyter



Desarrollo Web

Django, FastAPI, Flask, SQL, APIs REST



Automatización

Scripts de sistema, Selenium, scraping, openpyxl



Inteligencia Artif.

TensorFlow, PyTorch, OpenAI API, NLP, Visión



DevOps / Cloud

Docker, AWS/Azure SDK, CI/CD